

Guía Didáctica de la Implementación Actual PROM-RBF, ECM y MAW-ECM (2D)

Basada en la implementación actual del proyecto
RVE_homogenization_NeoHookean_using_Kratos_2D_MAWECM

21 de mayo de 2026

Índice

1. Objetivo de esta guía	3
2. Mapa mental completo (de Stage0 a Stage8)	3
2.1. Pipeline conceptual	3
2.2. Números reales de tu corrida actual (2D)	4
3. Diccionario de símbolos (con tamaño)	4
3.1. Convención de notación (para no colisionar símbolos)	4
3.2. Campos físicos y discretización	4
3.3. POD y coordenadas reducidas	5
3.4. Operadores LS/master-slave	5
3.5. ECM/MAW	5
4. Stage0: espacio paramétrico y trayectorias estructuradas	5
4.1. Qué es el espacio paramétrico aquí	5
4.2. Malla estructurada	6
4.3. Grafo y Laplaciano	6
4.4. Mapeo de μ a deformación objetivo de Stage1	6
5. Stage1: FOM	6
6. Stage2a: POD de fluctuaciones	6
6.1. Separación affine + fluctuación	6
6.2. Construcción POD	7
7. Stage2b: LS master + descomposición maestro/esclavo	7
7.1. Paso 1: recuperar μ desde strain aplicado	7
7.2. Paso 2: ajuste LS $q_{pod} \rightarrow \mu$	7
7.3. Paso 3: base master en espacio físico	7
7.4. Paso 4: subespacio slave en POD	8
7.5. Mini ejemplo numérico (didáctico)	9
8. Stage3: dataset de regresión	9
9. Stage4: RBF para decoder	9
9.1. Qué aprende	9
9.2. Modelo RBF	9
9.3. Por qué “sparse”	10

10.PROM online: ecuaciones que realmente se resuelven	10
10.1. Ansatz de desplazamiento	10
10.2. Jacobiano cinemático reducido	10
10.3. Residual y tangente reducidos	10
10.4. Paso de Newton reducido	10
11.Stage6a: construcción de matrices ECM clásicas	10
11.1. Qué se construye exactamente	10
11.2. Bloque residual: de dónde sale Q_{res}	11
11.3. Receta exacta: del residual elemental r_e al vector proyectado $q_{s,e}$	11
11.4. Mini ejemplo de llenado (residual)	13
11.5. Bloque homogenización: de dónde sale C_{hom}	13
12.Stage6b: ECM clásico (independent/cascade/single)	13
12.1. Problema matemático ideal (lo que queremos)	13
12.2. Problema práctico (cómo lo resolvemos en código)	13
12.3. Cómo se resuelve numéricamente (funciones concretas)	14
12.4. Qué son exactamente A y b en cada modo	14
12.5. Qué sale en <code>ecm_weights_all.npz</code>	15
13.Stage6c: creación de malla HROM (.mdpa)	15
13.1. Qué hace	15
13.2. Por qué es clave	15
14.Stage7: HPRM-RBF clásico	15
14.1. Qué se hiperreduce y qué no	15
14.2. Fórmula operativa	16
14.3. Recordatorio de C_m y C_s (porque aparece en el código)	16
15.Stage8a: dataset estructurado MAW	16
15.1. Qué aporta Stage8a frente a Stage6a	16
15.2. Qué se guarda exactamente	16
15.3. Bloques por nodo (la pieza que luego usa Stage8b)	16
16.Stage8b: MAW-ECM (Fase 1 actual, sin grafo)	17
16.1. Qué problema resolvemos hoy	17
16.2. Variables y tamaños	17
16.3. Paso 0: bootstrap ECM fijo	17
16.4. Paso 1: construir restricciones locales	17
16.5. Paso 2: ciclo de pruning (estilo Joaquín, primera fase)	18
16.6. Paso 3: salida del pruning	26
16.7. Paso 4: RBF final de pesos adaptativos	26
16.8. Nota importante: “sin grafo” no significa “sin Laplaciano en disco”	27
17.Modos MAW actuales (implementación 2D)	27
17.1. <code>maw-mode=res_only</code>	27
17.2. <code>maw-mode=single_res_hom</code>	27
17.3. Aclaración práctica sobre Z_{union}	27
18.Stage8 online: qué se compara exactamente	28

19. Aclaraciones clave (las que más confunden)	28
19.1. “¿q_pod se usa para resolver online?”	28
19.2. “¿Qué es R_r ?”	28
19.3. “¿Por qué aparece K_old?”	28
19.4. “¿Qué significa single en ECM/MAW?”	28
20. Ejemplo mini de tamaños (con números tuyos)	28
20.1. Decoder	29
20.2. MAW data	29
21. Checklist práctico para depurar	29
22. Conclusión	29

1. Objetivo de esta guía

Esta guía está escrita para alguien que **empieza desde cero** y quiere entender:

- qué significa cada símbolo;
- qué tamaño tiene cada matriz/vector;
- cómo se conectan los *stages* 0–8;
- qué hace exactamente ECM clásico;
- qué hace exactamente MAW–ECM;
- qué cambia en el modo nuevo `single_res_hom`.

La idea es que puedas explicarlo a otro estudiante de máster paso a paso, sin saltos.

2. Mapa mental completo (de Stage0 a Stage8)

2.1. Pipeline conceptual

1. **Stage0**: defines malla estructurada en espacio paramétrico μ y dos trayectorias.
2. **Stage1**: corres FOM en esas trayectorias y guardas snapshots.
3. **Stage2a**: construyes POD en DOFs libres (*fluctuaciones*).
4. **Stage2b**: construyes coordenadas maestro-esclavo (LS + descomposición ortogonal).
5. **Stage3**: armas dataset final de regresión y topología estructurada.
6. **Stage4**: entrenas RBF para el decoder $q_m \mapsto q_s$ (o variantes).
7. **Stage5**: validas PROM-RBF online contra FOM.
8. **Stage6**: construyes ECM clásico (residual y homogenización).
9. **Stage7**: validas HPROM-RBF clásico.
10. **Stage8a**: construyes dataset estructurado MAW (residual, y homogenización opcional).
11. **Stage8b**: entrenas MAW-ECM (fase actual: pruning local sin grafo; modo residual o single residual+hom).
12. **Stage8 test**: validas HPROM-MAWECM-RBF online.

2.2. Números reales de tu corrida actual (2D)

En una corrida típica tuya:

- DOFs totales: 4244.
- DOFs libres: 3924.
- DOFs Dirichlet: 320.
- snapshots FOM: 5574.
- dimensión POD: $r = 13$.
- dimensión master: $d_m = 2$.
- dimensión slave: $d_s = 11$.
- elementos FE: $n_e = 990$.
- nodos estructurados Stage0/MAW: $N_n = 451$.

3. Diccionario de símbolos (con tamaño)

3.1. Convención de notación (para no colisionar símbolos)

En esta guía voy a usar **siempre** esta convención:

- **Pesos ECM/MAW:** w (y matrices W_{train} cuando hay muchos nodos).
- **Matriz de snapshots:** X_{snap} (nunca W).
- **Adyacencia de grafo:** A_{adj} (nunca W).
- **Diagonal de grados:** D_{deg} (nunca D a secas).
- **Área de elemento:** $|\Omega_e|$ (no A_e para evitar choque con otras A).
- **Matrices tipo “A”:** siempre con subíndice explícito: A_m (master), A_j (restricción local MAW), A_{adj} (grafo).

3.2. Campos físicos y discretización

- $\mathbf{u} \in \mathbb{R}^{n_{dof}}$: desplazamientos globales.
- $\mathbf{u}_f \in \mathbb{R}^{n_f}$: desplazamientos en DOFs libres.
- $\mathbf{u}_d \in \mathbb{R}^{n_d}$: desplazamientos en DOFs fijos (Dirichlet).
- $n_{dof} = n_f + n_d$.
- $\mathbf{R}_f(\mathbf{u}) \in \mathbb{R}^{n_f}$: residual en espacio libre.
- $\mathbf{K}_{ff}(\mathbf{u}) \in \mathbb{R}^{n_f \times n_f}$: tangente libre-libre.

3.3. POD y coordenadas reducidas

- $\Phi_{rom} \in \mathbb{R}^{n_f \times r}$: base POD total.
- $\mathbf{q}_{pod} \in \mathbb{R}^r$: coordenadas POD.
- $\Phi_m \in \mathbb{R}^{n_f \times d_m}$: base master en desplazamientos.
- $\Phi_s \in \mathbb{R}^{n_f \times d_s}$: base slave en desplazamientos.
- $\mathbf{q}_m \in \mathbb{R}^{d_m}$: coordenadas master.
- $\mathbf{q}_s \in \mathbb{R}^{d_s}$: coordenadas slave.
- $d_m + d_s = r$ (en tu caso $2 + 11 = 13$).

3.4. Operadores LS/master-slave

- $\mathbf{T}_m \in \mathbb{R}^{d_m \times r}$: mapa LS desde $\mathbf{q}_{pod}$ a objetivo master.
- $\mathbf{A}_m \in \mathbb{R}^{d_m \times d_m}$: factor interno de la base master, obtenido de la SVD de $\Phi_c = \Phi_{rom} \mathbf{T}_m^\top$ (sección Stage2b).
- $\mathbf{C}_m \in \mathbb{R}^{r \times d_m}$: base master en espacio POD.
- $\mathbf{C}_s \in \mathbb{R}^{r \times d_s}$: base slave en espacio POD.

3.5. ECM/MAW

- n_e : número de elementos.
- $\mathcal{Z}$: conjunto de elementos seleccionados.
- $\mathbf{w} \in \mathbb{R}^{|\mathcal{Z}|}$: pesos ECM en soporte seleccionado.
- $\mathbf{w}^{full} \in \mathbb{R}^{n_e}$: mismos pesos embebidos en malla completa.
- $\mathbf{Q}_{ecm} \in \mathbb{R}^{(n_q N_s^{res}) \times n_e}$: bloques residual proyectado.
- $\mathbf{b}_{res} \in \mathbb{R}^{n_q N_s^{res}}$: suma completa residual.
- $\mathbf{C}_{hom} \in \mathbb{R}^{(6N_s^{hom}) \times n_e}$: bloques homogenización.
- $\mathbf{b}_{hom} \in \mathbb{R}^{6N_s^{hom}}$: suma completa homogenización.
- $\mathbf{A}_{adj} \in \mathbb{R}^{N_n \times N_n}$: adyacencia del grafo estructurado.
- $\mathbf{D}_{deg} \in \mathbb{R}^{N_n \times N_n}$: diagonal de grados del grafo estructurado.
- $\mathbf{K}_{graph} = \mathbf{D}_{deg} - \mathbf{A}_{adj} \in \mathbb{R}^{N_n \times N_n}$: Laplaciano usado en MAW.
- $\mathbf{A}_j \in \mathbb{R}^{m_j \times n_c}$: operador local MAW en nodo estructurado j .
- $\mathbf{b}_j \in \mathbb{R}^{m_j}$: lado derecho local MAW en nodo estructurado j .

4. Stage0: espacio paramétrico y trayectorias estructuradas

4.1. Qué es el espacio paramétrico aquí

En esta implementación, el espacio paramétrico base es:

$$\mu = [G_x, G_{xy}]$$

donde G_x y G_{xy} parametrizan una deformación macroscópica 2D (en este caso, rama triangular superior de F por construcción del script).

4.2. Malla estructurada

Se crea una malla cartesiana en μ :

$$G_x \in [G_x^{min}, G_x^{max}], \quad G_{xy} \in [G_{xy}^{min}, G_{xy}^{max}]$$

con $n_{gx} \times n_{gxy}$ nodos.

En tu configuración usual:

$$n_{gx} = 41, \quad n_{gxy} = 11, \quad N_n = 41 \cdot 11 = 451.$$

4.3. Grafo y Laplaciano

Se construye un grafo de vecindad sobre esa malla:

- nodo = punto estructurado en μ ;
- arista = vecino horizontal o vertical.

Con eso:

$$K_{graph} = D_{deg} - A_{adj},$$

donde A_{adj} es adyacencia y D_{deg} es diagonal de grados.

Importante: este grafo se guarda para análisis y para una fase 2 con acoplamiento espacial. En la fase 1 actual de MAW, el pruning se ejecuta sin acoplamiento por grafo.

4.4. Mapeo de μ a deformación objetivo de Stage1

Se guardan waypoints de strain $[E_{xx}, E_{yy}, \gamma_{xy}]$ usando:

$$E_{xx} = G_x + \frac{1}{2}G_x^2, \tag{1}$$

$$E_{yy} = \frac{1}{2}G_{xy}^2, \tag{2}$$

$$\gamma_{xy} = G_{xy}(1 + G_x), \tag{3}$$

cuando `mapping=green_lagrange_upper`.

5. Stage1: FOM

Se recorren trayectorias y se resuelve el problema no lineal completo. Se guarda por snapshot:

- **u** completa;
- strain aplicado;
- variables de homogenización.

Detalle clave: el número de incrementos por trayectoria está escalado por `ref_steps` (por ejemplo 400), lo que controla robustez de convergencia.

6. Stage2a: POD de fluctuaciones

6.1. Separación affine + fluctuación

Para cada snapshot:

$$\mathbf{u}_f = \mathbf{u}_{aff,f}(E) + \mathbf{u}_{fluc},$$

donde $\mathbf{u}_{aff,f}(E)$ se calcula desde la parte macroscópica y $\mathbf{u}_{fluc}$ es la fluctuación.

6.2. Construcción POD

Se apilan fluctuaciones:

$$X_{snap} \in \mathbb{R}^{N_s \times n_f}, \quad X_{snap}^\top \in \mathbb{R}^{n_f \times N_s}.$$

Se hace SVD:

$$X_{snap}^\top = U \Sigma V^\top,$$

y se toma $\Phi_{rom} = U_{(:,1:r)}$.

Entonces:

$$\mathbf{q}_{pod} = \Phi_{rom}^\top \mathbf{u}_{fluc}, \quad (\text{equivalente por filas a } Q_{pod} = X_{snap} \Phi_{rom}).$$

En tu caso: $r = 13$.

7. Stage2b: LS master + descomposición maestro/esclavo

7.1. Paso 1: recuperar μ desde strain aplicado

Desde E_{xx}, γ_{xy} se invierte el mapeo para obtener $[G_x, G_{xy}]$ (o $[F_{11}, F_{12}]$).

7.2. Paso 2: ajuste LS $q_{pod} \rightarrow \mu$

Con muestras:

$$Q \in \mathbb{R}^{N_s \times r}, \quad M \in \mathbb{R}^{N_s \times d_m} \quad (d_m = 2).$$

Se resuelve:

$$\min_{T_m} \|Q T_m^\top - M\|_F^2,$$

guardando T_m .

¿Cómo se resuelve $\min_{T_m}$ en el código?

En `stage2b_build_ls_master_from_pod.py` se usa:

$$\text{np.linalg.lstsq}(Q, M, \text{rcond=None}),$$

que computa una solución de mínimos cuadrados por SVD/pseudo-inversa. En forma compacta:

$$T_m^\top = Q^+ M,$$

donde Q^+ es la pseudo-inversa de Moore–Penrose. Si Q tuviera rango columna completo, esto coincide con:

$$T_m^\top = (Q^\top Q)^{-1} Q^\top M.$$

Importante: aquí no se hace una regularización explícita tipo ridge; la robustez viene del solver SVD interno de `lstsq`.

7.3. Paso 3: base master en espacio físico

Con $T_m^\top \in \mathbb{R}^{r \times d_m}$:

$$\Phi_c = \Phi_{rom} T_m^\top \in \mathbb{R}^{n_f \times d_m}.$$

SVD de Φ_c :

$$\Phi_c = U_m \Sigma_m V_m^\top.$$

$$\Phi_m := U_m(:, 1:d_m), \quad A_m := \Sigma_m V_m^\top.$$

Así, por construcción:

$$\Phi_m \in \mathbb{R}^{n_f \times d_m}, \quad A_m \in \mathbb{R}^{d_m \times d_m}.$$

De aquí sale exactamente A_m . No es un parámetro libre: es el factor derecho de la SVD truncada de Φ_c .

7.4. Paso 4: subespacio slave en POD

Se define:

$$C_m = \Phi_{rom}^\top \Phi_m \in \mathbb{R}^{r \times d_m}.$$

Se construye su complemento ortonormal:

$$C_s \in \mathbb{R}^{r \times d_s}, \quad d_s = r - d_m.$$

¿De dónde sale exactamente C_s ? (matemática + código)

La idea es: C_s debe generar las direcciones POD que **no** están en el subespacio master.

Paso A: tomas C_m y formas el proyector ortogonal al subespacio master:

$$P_\perp = I_r - C_m C_m^\top \in \mathbb{R}^{r \times r}.$$

Si C_m es ortonormal por columnas, entonces:

$$C_m^\top C_m = I_{d_m},$$

y $P_\perp$ proyecta a la parte “slave” del espacio POD.

Paso B: calculas autodescomposición de $P_\perp$:

$$P_\perp v_i = \lambda_i v_i.$$

Los autovectores con autovalores cercanos a 1 span-ean el complemento ortogonal. Como $d_s = r - d_m$, tomas exactamente d_s vectores:

$$C_s = [v_1, \dots, v_{d_s}] \in \mathbb{R}^{r \times d_s}.$$

Paso C (numérico): re-ortonormalizas C_s con QR para robustez:

$$C_s \leftarrow \text{qr}(C_s),$$

de modo que en la práctica se cumpla:

$$C_s^\top C_s \approx I_{d_s}, \quad C_m^\top C_s \approx 0.$$

Esto coincide con el código de `stage2b_build_ls_master_from_pod.py`:

- construye `proj_perp = I - C_m @ C_m.T`;
- usa `np.linalg.eigh(proj_perp)`;
- ordena autovectores y toma los primeros d_s ;
- aplica `np.linalg.qr` para ortonormalizar.

Interpretación práctica:

- $q_{pod} C_m$ = coordenadas de la parte master dentro del POD;
- $q_{pod} C_s$ = coordenadas de la parte slave dentro del POD;
- juntas reconstruyen las fluctuaciones POD sin perder información del subespacio reducido de rango r .

Para cada snapshot:

$$q_{m,proj} = q_{pod} C_m, \quad q_s = q_{pod} C_s.$$

Y para consistencia exacta con A_m :

$$q_m^{cons} = A_m^{-1} q_{m,proj}.$$

Muy importante (tu duda recurrente):

- el q_m usado en Stage3+online es q_m^{cons} (no el LS “crudo”);
- por eso el decoder queda coherente con $q_{master} = A_m q_m$ y con la reconstrucción.

7.5. Mini ejemplo numérico (didáctico)

Supón:

$$r = 4, \quad d_m = 2, \quad d_s = 2.$$

Para un snapshot:

$$q_{pod} = [0, 8, -0, 2, 0, 3, 1, 1].$$

Si

$$C_m = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad C_s = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix},$$

entonces:

$$q_{m,proj} = q_{pod} C_m = [0, 8, -0, 2], \quad q_s = q_{pod} C_s = [0, 3, 1, 1].$$

Si además

$$A_m = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix},$$

entonces:

$$q_m^{cons} = A_m^{-1} q_{m,proj} = [0, 4, -0, 2].$$

Y se cumple:

$$A_m q_m^{cons} = q_{m,proj}.$$

8. Stage3: dataset de regresión

Se empaqueta:

- entradas μ ;
- targets (en tu configuración actual: q_s);
- arrays completos q_m, q_s, q_{pod} ;
- matrices A_m, C_m, C_s ;
- topología estructurada Stage0 (nodos, quads, grafo, Laplaciano);
- mapeo nodo estructurado $\leftrightarrow$ snapshot más cercano.

Tu corrida: 5574 muestras; split 80/10/10.

9. Stage4: RBF para decoder

9.1. Qué aprende

En el flujo actual, se usa:

$$\text{input} = q_m \in \mathbb{R}^2, \quad \text{target} = q_s \in \mathbb{R}^{11}.$$

O sea, aprende:

$$N_{slave} : q_m \mapsto q_s.$$

9.2. Modelo RBF

Con centros c_ℓ :

$$\phi_\ell(q) = \exp\left(-\frac{1}{2} \sum_{a=1}^{d_m} \frac{(q_a - c_{\ell a})^2}{\ell_a^2}\right).$$

La salida es multi-output (11 componentes), con regularización y posible término polinomial.

9.3. Por qué “sparse”

No se usan necesariamente todos los puntos de entrenamiento como centros. Se seleccionan puntos inducidos (*inducing points*) para balancear costo/precisión.

10. PROM online: ecuaciones que realmente se resuelven

En Stage5/7/8, el solver PROM-RBF no “solo predice”: hace corrector de Newton en q_m .

10.1. Ansatz de desplazamiento

$$\mathbf{u}_f(q_m) = \mathbf{u}_{aff,f}(E) + \Phi_m(A_m q_m) + \Phi_s q_s(q_m).$$

10.2. Jacobiano cinemático reducido

$$\frac{\partial \mathbf{u}_f}{\partial q_m} = \Phi_m A_m + \Phi_s \frac{\partial q_s}{\partial q_m},$$

donde $\partial q_s / \partial q_m$ se calcula de forma **analítica** a partir del modelo RBF (derivando kernel y parte polinómica local; en modo **neighbors**, usando el vecindario activo).

10.3. Residual y tangente reducidos

$$r_r(q_m) = \left(\frac{\partial \mathbf{u}_f}{\partial q_m} \right)^\top R_f(\mathbf{u}_f(q_m)),$$
$$K_r(q_m) = \left(\frac{\partial \mathbf{u}_f}{\partial q_m} \right)^\top K_{ff}(\mathbf{u}_f(q_m)) \left(\frac{\partial \mathbf{u}_f}{\partial q_m} \right).$$

10.4. Paso de Newton reducido

$$K_r \Delta q_m = r_r, \quad q_m \leftarrow q_m + \Delta q_m.$$

Nota de convergencia clave: se monitorea $\|R_r\|$ (no $\|R_f\|$) para criterio reducido. Esto explica por qué puede verse R_f grande pero solver reducido estable.

11. Stage6a: construcción de matrices ECM clásicas

11.1. Qué se construye exactamente

En Stage6a no se “entrena” aún el ECM. Aquí solo se construyen las matrices que definen el problema de cubatura.

- Entrada: contribuciones por elemento en muchos estados de entrenamiento.
- Salida: matrices densas por bloques:

$$Q_{res}, b_{res}, C_{hom}, b_{hom}.$$

- Estas matrices quedan guardadas en: `Q_ecm.dat`, `b_full.dat`, `C_hom.dat`, `b_hom.dat`.

11.2. Bloque residual: de dónde sale Q_{res}

Para cada estado (snapshot) $s = 1, \dots, N_s^{res}$ y elemento $e = 1, \dots, n_e$, se calcula un vector

$$q_{s,e} \in \mathbb{R}^{n_q},$$

donde n_q es el número de ecuaciones reducidas que quieres reproducir (en tu caso típico, $n_q = 13$).

La interpretación de $q_{s,e}$ es:

- componente a componente, cuánto aporta **el elemento** e a cada ecuación reducida del residual en el estado s .

11.3. Receta exacta: del residual elemental r_e al vector proyectado $q_{s,e}$

Esta es la parte que normalmente queda implícita, pero aquí va explícita tal como está implementada en `stage6a_build_ecm_dataset.py`.

Paso A: residual elemental local

Para un estado s y elemento e , Kratos devuelve:

$$r_e^{(s)} \in \mathbb{R}^{n_{loc}(e)},$$

con

$$r_e^{(s)} = \text{elem.CalculateRightHandSide}(\dots).$$

Este vector está en el orden local de DOFs del elemento.

Paso B: quedarse solo con DOFs que realmente participan

El elemento también da su vector de IDs de ecuación local:

$$\text{id}_e \in \mathbb{Z}^{n_{loc}(e)}, \quad \text{id}_{e,a} = \text{EquationIdVector}[a].$$

- Si $\text{id}_{e,a} < 0$, ese DOF no participa en el sistema lineal global $\Rightarrow$ se descarta.
- Si $\text{id}_{e,a} \geq 0$ pero es DOF de Dirichlet (no libre), también se descarta para la proyección residual.

Define entonces el conjunto de índices locales válidos:

$$\mathcal{I}_e^{free} = \{a \in \{1, \dots, n_{loc}(e)\} : \text{id}_{e,a} \geq 0, \text{ dof libre}\}.$$

Con eso:

$$r_{e,free}^{(s)} \in \mathbb{R}^{n_{ef}}, \quad n_{ef} = |\mathcal{I}_e^{free}|,$$

tomando las entradas de $r_e^{(s)}$ en $\mathcal{I}_e^{free}$.

Paso C: proyector reducido local del elemento

Sea

$$\Phi_f \in \mathbb{R}^{n_f \times n_q}$$

la base POD de DOFs libres (archivo `pod_basis_free.npy`).

Para cada índice local $a \in \mathcal{I}_e^{free}$, su DOF global libre asociado es $p(a) \in \{1, \dots, n_f\}$ (en código: `map_g2f`).

Construimos la submatriz:

$$\Phi_{e,free} \in \mathbb{R}^{n_{ef} \times n_q}, \quad \Phi_{e,free}(a', i) = \Phi_f(p(a'), i),$$

y su traspuesta:

$$\Phi_{e,free}^\top \in \mathbb{R}^{n_q \times n_{ef}}.$$

Paso D: proyección residual por elemento

La contribución proyectada del elemento e en el estado s es:

$$q_{s,e} = \Phi_{e,free}^\top r_{e,free}^{(s)} \in \mathbb{R}^{n_q}.$$

Componente a componente:

$$q_{s,e}^{(i)} = \sum_{a'=1}^{n_{ef}} \Phi_{e,free}(a', i) r_{e,free,a'}^{(s)}.$$

Esto es exactamente el “residuo proyectado por elemento” que luego entra a ECM.

Forma equivalente con operadores de restricción

Si defines una matriz de selección local-global

$$R_e \in \{0, 1\}^{n_{ef} \times n_f},$$

entonces

$$\Phi_{e,free} = R_e \Phi_f, \quad q_{s,e} = (R_e \Phi_f)^\top r_{e,free}^{(s)}.$$

Cómo se arma la fila-bloque de Q_{res}

Para un estado s :

$$Q_{res}^{(s)} = [q_{s,1} \quad q_{s,2} \quad \cdots \quad q_{s,n_e}] \in \mathbb{R}^{n_q \times n_e}.$$

Luego se apila verticalmente para todos los s :

$$Q_{res} = \begin{bmatrix} Q_{res}^{(1)} \\ Q_{res}^{(2)} \\ \vdots \\ Q_{res}^{(N_s^{res})} \end{bmatrix} \in \mathbb{R}^{(n_q N_s^{res}) \times n_e}.$$

Qué representa b_{res}

En cada estado:

$$b_{res}^{(s)} = \sum_{e=1}^{n_e} q_{s,e} = Q_{res}^{(s)} \mathbf{1}.$$

Globalmente:

$$b_{res} = Q_{res} \mathbf{1}.$$

Este b_{res} es la referencia “full integration” que ECM intenta reproducir con pocos elementos y pesos.

Nota importante sobre esta implementación

En este pipeline, la proyección de Stage6a usa la base fija Φ_f . Es decir, el operador de proyección offline es fijo en todos los snapshots. No se está usando aquí un Jacobiano de variedad dependiente del estado para construir Q_{res} . Eso es coherente con el código actual de Stage6a/Stage8a.

11.4. Mini ejemplo de llenado (residual)

Supón $N_s^{res} = 2$, $n_q = 2$, $n_e = 3$. Entonces Q_{res} tiene tamaño 4×3 . Si

$$q_{1,1} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad q_{1,2} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad q_{1,3} = \begin{bmatrix} 5 \\ 6 \end{bmatrix},$$

$$q_{2,1} = \begin{bmatrix} 7 \\ 8 \end{bmatrix}, \quad q_{2,2} = \begin{bmatrix} 9 \\ 10 \end{bmatrix}, \quad q_{2,3} = \begin{bmatrix} 11 \\ 12 \end{bmatrix},$$

entonces

$$Q_{res} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \\ 7 & 9 & 11 \\ 8 & 10 & 12 \end{bmatrix}, \quad b_{res} = Q_{res} \mathbf{1} = \begin{bmatrix} 9 \\ 12 \\ 27 \\ 30 \end{bmatrix}.$$

11.5. Bloque homogenización: de dónde sale C_{hom}

Para cada estado s y elemento e se forma un vector de 6 componentes:

$$c_{s,e} = \begin{bmatrix} |\Omega_e| \bar{\varepsilon}_{xx} \\ |\Omega_e| \bar{\varepsilon}_{yy} \\ |\Omega_e| \bar{\gamma}_{xy} \\ |\Omega_e| \bar{\sigma}_{xx} \\ |\Omega_e| \bar{\sigma}_{yy} \\ |\Omega_e| \bar{\sigma}_{xy} \end{bmatrix} \in \mathbb{R}^6.$$

Muy importante: cada componente ya está ponderada por medida del elemento ($|\Omega_e|$ en 2D). Por eso, al sumar por elementos, obtienes la contribución macroscópica.

Se arma:

$$C_{hom} \in \mathbb{R}^{(6N_s^{hom}) \times n_e}, \quad b_{hom} = C_{hom} \mathbf{1}.$$

Luego este bloque se separa en

$$C_\varepsilon \in \mathbb{R}^{(3N_s^{hom}) \times n_e}, \quad C_\sigma \in \mathbb{R}^{(3N_s^{hom}) \times n_e}$$

para tratar strain y stress por separado cuando conviene.

12. Stage6b: ECM clásico (independent/cascade/single)

12.1. Problema matemático ideal (lo que queremos)

Dados $A \in \mathbb{R}^{m \times n_e}$ y $b \in \mathbb{R}^m$, ECM busca:

$$\min_{w \in \mathbb{R}^{n_e}} \|w\|_0 \quad \text{sueto a} \quad \|Aw - b\|_2 \leq \tau \|b\|_2, \quad w \geq 0.$$

- w : pesos de cubatura por elemento.
- $\|w\|_0$: número de elementos activos (sparseza).
- $w \geq 0$: no negatividad (propiedad física/numérica deseada).

12.2. Problema práctico (cómo lo resolvemos en código)

Como el problema ideal es combinatorio, Stage6b usa esta secuencia:

1. **Compresión algebraica** de $A^\top$ con RSVD:

$$A^\top \approx U \Sigma V^\top,$$

y se retiene una base U truncada por tolerancia.

2. **Greedy ECM** sobre esa base para seleccionar soporte $\mathcal{Z}$ y pesos positivos.
3. **Reconstrucción full**: se crea $w_{full} \in \mathbb{R}^{n_e}$, poniendo ceros fuera de $\mathcal{Z}$.

12.3. Cómo se resuelve numéricamente (funciones concretas)

RSVD (en código)

En `stage6b_compute_ecm_weights.py` se llama:

```
RandomizedSingularValueDecomposition.Calculate( $A^\top$ , truncation_tolerance =  $\tau_{svd}$ ).
```

Se conserva la base U truncada y esa base es la entrada de ECM.

ECM clásico (en código)

Se instancia:

```
EmpiricalCubatureMethod(ECM_tolerance,...)
```

y luego:

```
SetUp(ResidualsBasis=U, constrain_sum_of_weights=True,...).
```

Ese `constrain_sum_of_weights=True` añade una restricción auxiliar de “conteo” para evitar la solución trivial y fijar una normalización global de pesos.

Paso greedy interno (detalle algebraico)

En el ECM clásico (`empirical_cubature_method.py`):

- selección de candidato:

$$i = \arg \max_{k \in \mathcal{Y}} g_k^\top r,$$

donde g_k es la columna candidata y r es residual del problema ECM;

- en la primera iteración:

$$\alpha = \text{np.linalg.lstsq}(G_{(:,i)}, b),$$

- en iteraciones siguientes, actualización rápida del inverso (Sherman-Morrison-like) con `_UpdateWeightsInverse` sin recomputar todo desde cero;
- si aparecen pesos negativos, se corrige active-set: se sacan esos índices del soporte y se reevalúan pesos.

Reconstrucción final

Si $\mathcal{Z}$ es soporte seleccionado y $w_{\mathcal{Z}}$ sus pesos:

$$w_{full}(e) = \begin{cases} w_{\mathcal{Z}}(k), & e = \mathcal{Z}_k, \\ 0, & e \notin \mathcal{Z}. \end{cases}$$

12.4. Qué son exactamente A y b en cada modo

Modo independent

Tres problemas separados:

$$A_{res} = Q_{res}, b_{res}; \quad A_{\varepsilon} = C_{\varepsilon}, b_{\varepsilon}; \quad A_{\sigma} = C_{\sigma}, b_{\sigma}.$$

Resultado: $(\mathcal{Z}_{res}, w_{res}), (\mathcal{Z}_{\varepsilon}, w_{\varepsilon}), (\mathcal{Z}_{\sigma}, w_{\sigma})$.

Modo cascade

Mismos A, b que **independent**, pero inicializando cada selección con el soporte previo ($\mathcal{Z}_{res} \rightarrow \mathcal{Z}_{\varepsilon} \rightarrow \mathcal{Z}_{\sigma}$) para favorecer consistencia.

Modo single

Un solo problema combinado:

$$\tilde{A} = \begin{bmatrix} \alpha_{res} Q_{res} \\ \alpha_\varepsilon C_\varepsilon \\ \alpha_\sigma C_\sigma \end{bmatrix}, \quad \tilde{b} = \begin{bmatrix} \alpha_{res} b_{res} \\ \alpha_\varepsilon b_\varepsilon \\ \alpha_\sigma b_\sigma \end{bmatrix}.$$

Los factores α normalizan bloques (por norma Frobenius, por fila o sin normalizar) para que un bloque no domine numéricamente al resto.

12.5. Qué sale en `ecm_weights_all.npz`

Las claves más importantes para entender el pipeline son:

$$Z_{res}, Z_\varepsilon, Z_\sigma, Z_{union}, \quad w_{res}, w_\varepsilon, w_\sigma, \quad w_{res}^{full}, w_\varepsilon^{full}, w_\sigma^{full}.$$

- $Z_\star$: índices de elementos seleccionados.
- $w_\star$: pesos solo del soporte.
- $w_\star^{full}$: vector largo de tamaño n_e con ceros fuera del soporte.

13. Stage6c: creación de malla HROM (.mdpa)

Aquí pasas de “soporte algebraico” a “malla física”.

13.1. Qué hace

1. Toma un conjunto de selección (por ejemplo `Z_union`).
2. Extrae nodos/elementos necesarios del `.mdpa` completo.
3. Escribe un `.mdpa` reducido para corridas HROM.
4. Guarda metadatos de mapeo `full ↔ hrom`.

13.2. Por qué es clave

Sin este paso, podrías tener pesos ECM correctos pero aún estar montando sistemas sobre la malla completa en el solver. Con este paso, el ensamblaje también se restringe a los elementos seleccionados.

14. Stage7: HPROM-RBF clásico

14.1. Qué se hiperreduce y qué no

Con ECM clásico:

- **Residual/Tangente**: se ensamblan en soporte ECM fijo.
- **Homogenización**: se hace con pesos fijos (w_ε, w_σ) o con full-mesh, según configuración.
- **Incógnita de Newton**: sigue siendo q_m .

14.2. Fórmula operativa

$$r_r^{hprom}(q_m) \approx \left(\frac{\partial u_f}{\partial q_m} \right)^\top \sum_{e \in \mathcal{Z}_{res}} w_e^{res} r_e(u(q_m)).$$

$$K_r^{hprom}(q_m) \approx \left(\frac{\partial u_f}{\partial q_m} \right)^\top \left(\sum_{e \in \mathcal{Z}_{res}} w_e^{res} K_e(u(q_m)) \right) \left(\frac{\partial u_f}{\partial q_m} \right).$$

Homogenización (ejemplo en stress):

$$\bar{\sigma}(q_m) \approx \sum_{e \in \mathcal{Z}_\sigma} w_e^\sigma \bar{\sigma}_e(q_m).$$

14.3. Recordatorio de C_m y C_s (porque aparece en el código)

Aunque en online resuelves en q_m , el código puede reconstruir:

$$q_{pod} = C_m(A_m q_m) + C_s q_s.$$

- C_m : mezcla la parte master hacia coordenada POD.
- C_s : mezcla la parte slave hacia coordenada POD.
- Ambos salen de un ajuste LS offline (Stage2b), no se “inventan” online.

15. Stage8a: dataset estructurado MAW

15.1. Qué aporta Stage8a frente a Stage6a

Stage6a construye un problema ECM **global**. Stage8a lo reordena por **nodo estructurado del manifold** para poder imponer restricciones locales $A_j w_j = b_j$ (una por cada estado estructurado j).

15.2. Qué se guarda exactamente

Con $N_n = 451$ nodos estructurados y $n_e = 990$ elementos:

$$Q_{res} \in \mathbb{R}^{(n_q N_n) \times n_e}, \quad b_{res} \in \mathbb{R}^{n_q N_n}.$$

En tu caso típico ($n_q = 13$):

$$Q_{res} \in \mathbb{R}^{(13 \cdot 451) \times 990}.$$

Si activas `-include-homogenization 1`, además:

$$C_{hom} \in \mathbb{R}^{(6N_n) \times n_e}, \quad b_{hom} \in \mathbb{R}^{6N_n}.$$

15.3. Bloques por nodo (la pieza que luego usa Stage8b)

Para cada nodo estructurado $j \in \{1, \dots, N_n\}$:

- bloque residual local:

$$Q_{res}^{(j)} \in \mathbb{R}^{n_q \times n_e};$$

- bloque de homogenización local (si está habilitado):

$$C_{hom}^{(j)} \in \mathbb{R}^{6 \times n_e}.$$

- coordenada de estado asociada:

$$q_m^{(j)} \in \mathbb{R}^{d_m}.$$

16. Stage8b: MAW-ECM (Fase 1 actual, sin grafo)

16.1. Qué problema resolvemos hoy

En la fase 1 actual no estamos usando acoplamiento por grafo en el pruning (`-use-global-graph-2ndstage 0`). El objetivo es reducir soporte manteniendo factibilidad local exacta:

$$A_j w^{(j)} = b_j, \quad w^{(j)} \geq 0, \quad j = 1, \dots, N_n.$$

16.2. Variables y tamaños

- Soporte candidato inicial:

$$\mathcal{Z}_{ini}, \quad n_c := |\mathcal{Z}_{ini}|.$$

- Pesos adaptativos por nodo estructurado:

$$W_{adapt} = \begin{bmatrix} | & | & & | \\ w^{(1)} & w^{(2)} & \dots & w^{(N_n)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n_c \times N_n}.$$

- Conjunto activo de filas/candidatos:

$$I_{activo} \subseteq \{1, \dots, n_c\}.$$

Nota de lectura de código: en scripts MATLAB/Python este conjunto suele aparecer como `iCAND`.

16.3. Paso 0: bootstrap ECM fijo

Primero se genera una regla fija inicial factible:

$$(\mathcal{Z}_{ini}, w_{ini}).$$

Según el modo:

- `maw-mode=res_only`: bootstrap desde Q_{res} .
- `maw-mode=single_res_hom`: bootstrap sobre bloque combinado $[Q_{res}; C_\varepsilon; C_\sigma]$.

Luego se inicializa:

$$w^{(j)} \equiv w_{ini} \quad \forall j.$$

16.4. Paso 1: construir restricciones locales

Modo `res_only`

$$A_j = Q_{res}^{(j)}(:, \mathcal{Z}_{ini}) \in \mathbb{R}^{n_q \times n_c}, \quad b_j = A_j w_{ini} \in \mathbb{R}^{n_q}.$$

Modo `single_res_hom`

$$A_j = \begin{bmatrix} Q_{res}^{(j)}(:, \mathcal{Z}_{ini}) \\ C_{hom}^{(j)}(:, \mathcal{Z}_{ini}) \end{bmatrix} \in \mathbb{R}^{(n_q+6) \times n_c}, \quad b_j = A_j w_{ini}.$$

Restricción opcional de suma de pesos

Si activas `-enforce-sum-weights 1`, a cada bloque local se le agrega una fila:

$$\mathbf{1}^\top w^{(j)} = s_{target}.$$

Es decir:

$$\tilde{A}_j = \begin{bmatrix} A_j \\ \mathbf{1}^\top \end{bmatrix}, \quad \tilde{b}_j = \begin{bmatrix} b_j \\ s_{target} \end{bmatrix}.$$

Esto aumenta rigidez del problema local y puede imponer un piso en el soporte mínimo alcanzable.

16.5. Paso 2: ciclo de pruning (estilo Joaquín, primera fase)

Paso 2 explicado con tus números (153 candidatos, 451 nodos)

En tu caso:

$$n_c = 153, \quad N_n = 451, \quad n_q = 13.$$

Al terminar el Paso 1 tienes, para cada nodo j :

$$A_j \in \mathbb{R}^{13 \times 153}, \quad b_j \in \mathbb{R}^{13},$$

y una inicialización global:

$$W_{adapt} \in \mathbb{R}^{153 \times 451}, \quad W_{adapt}(:, j) = w_{ini} \quad \forall j.$$

El objetivo del Paso 2 es quitar columnas/candidatos de forma **común** para todos los nodos, sin perder factibilidad local.

Iteración típica del pruning:

1. Estado actual: $k = |I_{activo}|$ candidatos vivos (al inicio $k = 153$).
2. Se propone eliminar un candidato $p \in I_{activo}$.
3. Queda conjunto tentativo:

$$I_{keep} = I_{activo} \setminus \{p\}, \quad |I_{keep}| = k - 1.$$

4. Para cada nodo $j = 1, \dots, 451$ se trabaja con:

$$A_{j,keep} = A_j(:, I_{keep}) \in \mathbb{R}^{13 \times (k-1)}.$$

5. Para ese nodo, con peso previo

$$w_{j,before} = W_{adapt}(I_{keep}, j) \in \mathbb{R}^{k-1},$$

se fuerza de nuevo la ecuación local:

$$A_{j,keep} w_{j,after} = b_j.$$

6. Se busca el cambio mínimo:

$$w_{j,after} = w_{j,before} + \Delta w_j, \quad A_{j,keep} \Delta w_j = b_j - A_{j,keep} w_{j,before}.$$

En código (NOENF) se resuelve con:

$$\Delta w_j = \text{np.linalg.lstsq}(A_{j,keep}, rhs).$$

7. Si para **algún** nodo falla rango o sale peso negativo, ese candidato p se rechaza.
8. Si para **todos** los nodos sale factible, se acepta:

$$I_{activo} \leftarrow I_{keep}, \quad W_{adapt}(p, :) = 0.$$

Esto se repite hasta no poder eliminar más o hasta alcanzar n_{stop} .

Qué significa “candidato común”

Es clave: no estás quitando un elemento para un nodo sí y otro no. Cuando eliminas p , lo eliminas para **todas** las 451 columnas de W_{adapt} . Por eso el soporte final es común y luego se puede usar en un único HROM mdpa.

Por qué puede estancarse

Aunque $k - 1 \geq 13$, puede pasar que:

- para algunos nodos $A_{j,keep}$ pierda rango efectivo;
- o la solución exacta local requiera entradas negativas.

Cuando eso pasa para muchos candidatos, el algoritmo ya no puede seguir podando.

NOENF vs NOENFe en una frase

- NOENF: intenta actualización rápida y rechaza si salen negativos.
- NOENFe: activa enforcement explícito de no-negatividad por active-set local.

Si NOENF no puede reducir más, NOENFe intenta rescatar más eliminaciones factibles.

Ejemplo numérico completo (matrices explícitas)

Para ver el mecanismo de verdad, toma un caso pequeño:

$$n_q = 2, \quad n_c = 4, \quad N_n = 3.$$

Soporte inicial:

$$\mathcal{Z}_{ini} = \{1, 2, 3, 4\}.$$

Peso fijo inicial:

$$w_{ini} = \begin{bmatrix} 0,5 \\ 0,7 \\ 0,6 \\ 0,4 \end{bmatrix}.$$

Definimos tres nodos estructurados con:

$$A_1 = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 0 & 2 & 2 \\ 0 & 1 & 1 & 1 \end{bmatrix}, \quad A_3 = \begin{bmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 1 \end{bmatrix}.$$

Y:

$$b_j = A_j w_{ini}.$$

Entonces:

$$b_1 = \begin{bmatrix} 1,5 \\ 1,7 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix}, \quad b_3 = \begin{bmatrix} 2,0 \\ 2,4 \end{bmatrix}.$$

Paso 0 (ejemplo pequeño): cómo se elige el candidato Aquí la decisión **no** se toma con un único w_{ini} , sino con la matriz adaptativa actual:

$$W_{adapt}^{(t)} \in \mathbb{R}^{n_c \times N_n}.$$

En este ejemplo:

$$n_c = 4, \quad N_n = 3.$$

Al inicio:

$$W_{adapt}^{(0)} = \begin{bmatrix} 0,5 & 0,5 & 0,5 \\ 0,7 & 0,7 & 0,7 \\ 0,6 & 0,6 & 0,6 \\ 0,4 & 0,4 & 0,4 \end{bmatrix},$$

porque en $t = 0$ todas las columnas son w_{ini} .

Se calcula la actividad por candidato:

$$a_i^{(t)} = \sum_{j=1}^{N_n} W_{adapt}^{(t)}(i, j).$$

Entonces:

$$a^{(0)} = \begin{bmatrix} 1,5 \\ 2,1 \\ 1,8 \\ 1,2 \end{bmatrix},$$

y el orden ascendente es:

$$4 < 1 < 3 < 2.$$

Por eso el **primer intento real** es eliminar el candidato 4.

Intento A: eliminar candidato 4 (se acepta) Queda:

$$I_{keep} = \{1, 2, 3\}.$$

Matrices literales de este intento:

$$A_{1,keep} = A_1(:, \{1, 2, 3\}) = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix},$$

$$A_{2,keep} = A_2(:, \{1, 2, 3\}) = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix},$$

$$A_{3,keep} = A_3(:, \{1, 2, 3\}) = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \end{bmatrix}.$$

Pesos “before” literales:

$$w_{1,before} = w_{2,before} = w_{3,before} = \begin{bmatrix} 0,5 \\ 0,7 \\ 0,6 \end{bmatrix}.$$

Definiciones (explícitas) para este intento:

- $A_{j,keep} \in \mathbb{R}^{n_q \times |I_{keep}|}$: submatriz de A_j tomando solo las columnas de índices en I_{keep} . En este ejemplo, $n_q = 2$ y $|I_{keep}| = 3$, por tanto $A_{j,keep} \in \mathbb{R}^{2 \times 3}$.
- $w_{j,before} \in \mathbb{R}^{|I_{keep}|}$: vector de pesos del nodo j antes de intentar eliminar el candidato, restringido a las filas activas I_{keep} .
- $\Delta w_j \in \mathbb{R}^{|I_{keep}|}$: incremento que corrige $w_{j,before}$ para mantener exactitud local.
- $w_{j,after} = w_{j,before} + \Delta w_j$: pesos locales tras el intento.

En cada nodo:

$$A_{j,keep} \Delta w_j = b_j - A_{j,keep} w_{j,before}, \quad w_{j,after} = w_{j,before} + \Delta w_j.$$

Definimos para hacerlo ultra explícito:

$$y_{j,before} := A_{j,keep} w_{j,before}, \quad y_{j,after} := A_{j,keep} w_{j,after}.$$

Con $w_{j,before} = [0,5,0,7,0,6]^\top$, la solución de mínimo cambio da:

$$\Delta w_1 = \begin{bmatrix} 0,1333 \\ 0,1333 \\ 0,2667 \end{bmatrix}, \quad \Delta w_2 = \begin{bmatrix} 0,1333 \\ 0,0667 \\ 0,3333 \end{bmatrix}, \quad \Delta w_3 = \begin{bmatrix} 0,1333 \\ 0,1333 \\ 0,1333 \end{bmatrix}.$$

Por tanto:

$$w_{1,after} = \begin{bmatrix} 0,6333 \\ 0,8333 \\ 0,8667 \end{bmatrix}, \quad w_{2,after} = \begin{bmatrix} 0,6333 \\ 0,7667 \\ 0,9333 \end{bmatrix}, \quad w_{3,after} = \begin{bmatrix} 0,6333 \\ 0,8333 \\ 0,7333 \end{bmatrix}.$$

Todos son no-negativos y satisfacen $A_{j,keep} w_{j,after} = b_j$, así que se acepta la eliminación del 4.

$$y_{1,before} = \begin{bmatrix} 1,1 \\ 1,3 \end{bmatrix}, \quad y_{1,after} = \begin{bmatrix} 1,5 \\ 1,7 \end{bmatrix}, \quad y_{2,before} = \begin{bmatrix} 1,7 \\ 1,3 \end{bmatrix}, \quad y_{2,after} = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix}, \quad y_{3,before} = \begin{bmatrix} 2,0 \\ 2,7 \end{bmatrix}, \quad y_{3,after} = \begin{bmatrix} 2,0 \\ 2,4 \end{bmatrix}.$$

La nueva matriz adaptativa queda:

$$W_{adapt}^{(1)} = \begin{bmatrix} 0,6333 & 0,6333 & 0,6333 \\ 0,8333 & 0,7667 & 0,8333 \\ 0,8667 & 0,9333 & 0,7333 \\ 0 & 0 & 0 \end{bmatrix}.$$

Ahora:

$$a^{(1)} = \begin{bmatrix} 1,900 \\ 2,4333 \\ 2,5333 \\ 0 \end{bmatrix},$$

de modo que, sobre $I_{positivo}^{(1)} = \{1, 2, 3\}$, el siguiente candidato real es el 1 (no el 2).

Mismo Paso 1 pero imponiendo $\mathbf{1}^\top w = S_{target}$ Ahora repetimos **solo** el intento A (eliminar 4), pero añadiendo:

$$\mathbf{1}^\top w_{j,after} = S_{target}.$$

Para este ejemplo tomamos:

$$S_{target} = 2,2 = \mathbf{1}^\top \begin{bmatrix} 0,5 \\ 0,7 \\ 0,6 \\ 0,4 \end{bmatrix},$$

que es la suma total inicial de pesos en el soporte de 4 candidatos.

Entonces, para cada nodo:

$$\tilde{A}_{j,keep} w_{j,after} = \tilde{b}_j, \quad \tilde{A}_{j,keep} = \begin{bmatrix} A_{j,keep} \\ 1 \ 1 \ 1 \end{bmatrix}.$$

Nodo 1:

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 1,5 \\ 1,7 \\ 2,2 \end{bmatrix} \Rightarrow w_{1,after}^{sum} = \begin{bmatrix} 0,5 \\ 0,7 \\ 1,0 \end{bmatrix}.$$

Nodo 2:

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2,5 \\ 1,7 \\ 2,2 \end{bmatrix} \Rightarrow w_{2,after}^{sum} = \begin{bmatrix} 0,5 \\ 0,7 \\ 1,0 \end{bmatrix}.$$

Nodo 3:

$$\begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2,0 \\ 2,4 \\ 2,2 \end{bmatrix}.$$

La tercera ecuación aquí es combinación lineal de las dos primeras, y el mínimo cambio da:

$$w_{3,after}^{sum} = \begin{bmatrix} 0,6333 \\ 0,8333 \\ 0,7333 \end{bmatrix}.$$

Con esto:

$$W_{adapt}^{(1,sum)} = \begin{bmatrix} 0,5000 & 0,5000 & 0,6333 \\ 0,7000 & 0,7000 & 0,8333 \\ 1,0000 & 1,0000 & 0,7333 \\ 0 & 0 & 0 \end{bmatrix}.$$

Observa que para cada nodo se cumple:

$$\mathbf{1}^\top w_{j,after}^{sum} = 2,2.$$

Comparación directa del Paso 1:

- Sin restricción de suma:

$$W_{adapt}^{(1)} = \begin{bmatrix} 0,6333 & 0,6333 & 0,6333 \\ 0,8333 & 0,7667 & 0,8333 \\ 0,8667 & 0,9333 & 0,7333 \\ 0 & 0 & 0 \end{bmatrix}.$$

- Con $\mathbf{1}^\top w = S_{target}$:

$$W_{adapt}^{(1,sum)} = \begin{bmatrix} 0,5000 & 0,5000 & 0,6333 \\ 0,7000 & 0,7000 & 0,8333 \\ 1,0000 & 1,0000 & 0,7333 \\ 0 & 0 & 0 \end{bmatrix}.$$

Desde aquí, para no mezclar ramas, continuamos el ejemplo principal por la rama **sin** restricción de suma (la que ya veníamos usando).

Intento B: eliminar candidato 1 (se acepta) Ahora:

$$I_{keep} = \{2, 3\}.$$

Definiciones (mismas ideas, nuevo I_{keep}):

- $A_{j,keep}$: submatriz de A_j con columnas $\{2, 3\}$. Aquí $A_{j,keep} \in \mathbb{R}^{2 \times 2}$.
- $w_{j,before} \in \mathbb{R}^2$: columna j de $W_{adapt}^{(1)}$ restringida a filas $\{2, 3\}$.
- $\Delta w_j \in \mathbb{R}^2$ y $w_{j,after} = w_{j,before} + \Delta w_j$.

Para este intento, en cada nodo trabajamos con:

$$A_{j,keep} \Delta w_j = b_j - A_{j,keep} w_{j,before}, \quad w_{j,after} = w_{j,before} + \Delta w_j.$$

Y otra vez:

$$y_{j,before} := A_{j,keep} w_{j,before}, \quad y_{j,after} := A_{j,keep} w_{j,after}.$$

Aquí $w_{j,before}$ se toma de $W_{adapt}^{(1)}$ en las filas $\{2, 3\}$.

Nodo 1 ($j = 1$):

$$A_{1,keep} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 1,5 \\ 1,7 \end{bmatrix}$$

$$w_{1,before} = \begin{bmatrix} 0,8333 \\ 0,8667 \end{bmatrix}, \quad y_{1,before} = A_{1,keep} w_{1,before} = \begin{bmatrix} 0,8667 \\ 1,7000 \end{bmatrix},$$

$$RHS_1 = b_1 - A_{1,keep} w_{1,before} = \begin{bmatrix} 1,5 \\ 1,7 \end{bmatrix} - \begin{bmatrix} 0,8667 \\ 1,7000 \end{bmatrix} = \begin{bmatrix} 0,6333 \\ 0 \end{bmatrix}.$$

Resolviendo:

$$\Delta w_1 = \begin{bmatrix} -0,6333 \\ 0,6333 \end{bmatrix}, \quad w_{1,after} = \begin{bmatrix} 0,2000 \\ 1,5000 \end{bmatrix}.$$

Chequeo:

$$y_{1,after} = A_{1,keep} w_{1,after} = \begin{bmatrix} 1,5 \\ 1,7 \end{bmatrix} = b_1.$$

Nodo 2 ($j = 2$):

$$A_{2,keep} = \begin{bmatrix} 0 & 2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix}$$

$$w_{2,before} = \begin{bmatrix} 0,7667 \\ 0,9333 \end{bmatrix}, \quad y_{2,before} = A_{2,keep} w_{2,before} = \begin{bmatrix} 1,8666 \\ 1,7000 \end{bmatrix},$$

$$RHS_2 = b_2 - A_{2,keep} w_{2,before} = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix} - \begin{bmatrix} 1,8666 \\ 1,7000 \end{bmatrix} = \begin{bmatrix} 0,6334 \\ 0 \end{bmatrix}.$$

Resolviendo:

$$\Delta w_2 = \begin{bmatrix} -0,3167 \\ 0,3167 \end{bmatrix}, \quad w_{2,after} = \begin{bmatrix} 0,4500 \\ 1,2500 \end{bmatrix}.$$

Chequeo:

$$y_{2,after} = A_{2,keep} w_{2,after} = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix} = b_2.$$

Nodo 3 ($j = 3$):

$$A_{3,keep} = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2,0 \\ 2,4 \end{bmatrix}$$

$$w_{3,before} = \begin{bmatrix} 0,8333 \\ 0,7333 \end{bmatrix}, \quad y_{3,before} = A_{3,keep} w_{3,before} = \begin{bmatrix} 0,7333 \\ 2,3999 \end{bmatrix},$$

$$RHS_3 = b_3 - A_{3,keep} w_{3,before} = \begin{bmatrix} 2,0 \\ 2,4 \end{bmatrix} - \begin{bmatrix} 0,7333 \\ 2,3999 \end{bmatrix} \approx \begin{bmatrix} 1,2667 \\ 0 \end{bmatrix}.$$

Resolviendo:

$$\Delta w_3 = \begin{bmatrix} -0,6333 \\ 1,2667 \end{bmatrix}, \quad w_{3,after} = \begin{bmatrix} 0,2000 \\ 2,0000 \end{bmatrix}.$$

Chequeo:

$$y_{3,after} = A_{3,keep} w_{3,after} = \begin{bmatrix} 2,0 \\ 2,4 \end{bmatrix} = b_3.$$

Todos los $w_{j,after}$ son no-negativos, así que se acepta eliminar el candidato 1. La nueva matriz adaptativa es:

$$W_{adapt}^{(2)} = \begin{bmatrix} 0 & 0 & 0 \\ 0,2000 & 0,4500 & 0,2000 \\ 1,5000 & 1,2500 & 2,0000 \\ 0 & 0 & 0 \end{bmatrix}.$$

Luego:

$$a^{(2)} = \begin{bmatrix} 0 \\ 0,8500 \\ 4,7500 \\ 0 \end{bmatrix},$$

y en $I_{positivo}^{(2)} = \{2, 3\}$ el siguiente candidato natural es el 2.

Intento C: eliminar candidato 2 (se rechaza) Tras el intento B:

$$I_{activo} = \{2, 3\},$$

y el siguiente en ranking es el 2. Si lo quitas:

$$I_{keep} = \{3\}.$$

Definiciones en este caso:

- $A_{j,keep}$: submatriz de A_j con la única columna 3 ($A_{j,keep} \in \mathbb{R}^{2 \times 1}$).
- $w_{j,before} \in \mathbb{R}^1$: peso actual del candidato 3 en el nodo j .
- $\Delta w_j \in \mathbb{R}^1$: único incremento escalar a resolver.

Usando $W_{adapt}^{(2)}$, para nodo 2:

$$w_{2,before} = [1,2500], \quad A_{2,keep} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix}.$$

$$y_{2,before} = A_{2,keep} w_{2,before} = \begin{bmatrix} 2,5 \\ 1,25 \end{bmatrix}, \quad RHS_2 = b_2 - A_{2,keep} w_{2,before} = \begin{bmatrix} 0 \\ 0,45 \end{bmatrix}.$$

Con incremento escalar Δw_2 :

$$2(1,25 + \Delta w_2) = 2,5, \quad 1,25 + \Delta w_2 = 1,7.$$

Equivalentemente:

$$\Delta w_2 = 0, \quad \Delta w_2 = 0,45.$$

Incompatible. No existe un único Δw_2 que satisfaga ambas ecuaciones exactas. Por eso este candidato se rechaza.

La matriz queda sin cambios:

$$W_{adapt}^{(3)} = W_{adapt}^{(2)}.$$

Variante: imponer conservación de suma de pesos Si quieres imponer, en cada nodo j , que la suma de pesos se conserve, no cambias la lógica base: solo añades una ecuación lineal extra al sistema local.

Sistema base local:

$$A_{j,keep} w_{j,after} = b_j.$$

Sistema aumentado:

$$\tilde{A}_{j,keep} w_{j,after} = \tilde{b}_j, \quad \tilde{A}_{j,keep} = \begin{bmatrix} A_{j,keep} \\ c^\top \end{bmatrix}, \quad \tilde{b}_j = \begin{bmatrix} b_j \\ c^\top w_{j,ref} \end{bmatrix}.$$

Caso correcto para conservar suma:

$$c = \mathbf{1}.$$

Entonces:

$$\mathbf{1}^\top w_{j,after} = \mathbf{1}^\top w_{j,ref}.$$

Si en tu caso real quieres preservar 990, usas:

$$c = \mathbf{1}_{153}, \quad \mathbf{1}_{153}^\top w_{j,ref} = 990.$$

No es lo mismo usar $c = w_{ini}$:

$$w_{ini}^\top w_{j,after} = w_{ini}^\top w_{j,ref},$$

eso conserva un producto ponderado, no la suma.

Mini-cálculo explícito (ejemplo pequeño) Tomamos el nodo 2 del intento A con:

$$A_{2,keep} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix}, \quad w_{2,ref} = w_{2,before} = \begin{bmatrix} 0,5 \\ 0,7 \\ 0,6 \end{bmatrix}.$$

Si impones conservación de suma con $c = \mathbf{1}_3$:

$$\tilde{A}_{2,keep} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \tilde{b}_2 = \begin{bmatrix} 2,5 \\ 1,7 \\ 1,8 \end{bmatrix}$$

(porque $1,8 = 0,5 + 0,7 + 0,6$).

Resolviendo:

$$w_{2,after} = \begin{bmatrix} 0,1 \\ 0,5 \\ 1,2 \end{bmatrix}, \quad \Delta w_2 = \begin{bmatrix} -0,4 \\ -0,2 \\ 0,6 \end{bmatrix}.$$

Chequeos:

$$A_{2,keep} w_{2,after} = \begin{bmatrix} 2,5 \\ 1,7 \end{bmatrix} = b_2, \quad \mathbf{1}^\top w_{2,after} = 1,8 = \mathbf{1}^\top w_{2,ref}.$$

Si en cambio usas $c = w_{2,ref} = [0,5, 0,7, 0,6]^\top$:

$$w_{2,after} \approx \begin{bmatrix} 0,0636 \\ 0,4818 \\ 1,2182 \end{bmatrix},$$

y:

$$\mathbf{1}^\top w_{2,after} \approx 1,7636 \neq 1,8.$$

Con esto ves por qué, si el objetivo es conservar *suma*, la fila extra debe ser $\mathbf{1}^\top$ (o la medida geométrica que realmente quieras conservar).

Regla global de decisión (en el ejemplo pequeño y en el caso general):

- El ranking se hace con $W_{adapt}^{(t)}$ actual, no con w_{ini} fijo.
- Se acepta un candidato solo si *todos* los nodos son factibles.
- Si falla un nodo (incompatibilidad, rango o negatividad), el candidato se rechaza.

Qué aprendiste aquí

- Δw_j es vector para cada nodo j .
- Si apilas todos los nodos, obtienes un bloque de cambios

$$\Delta W = \begin{bmatrix} | & | & | \\ \Delta w_1 & \Delta w_2 & \Delta w_3 \\ | & | & | \end{bmatrix}.$$

- La decisión aceptar/rechazar un candidato es global: si falla un solo nodo, se rechaza ese candidato.

Sea

$$a_i := \sum_{j=1}^{N_n} W_{adapt}(i, j),$$

que interpretamos como **actividad total del candidato** i en todos los nodos del manifold. Se define:

$$I_{positivo} = \{i \in I_{activo} : a_i > 0\},$$

y se ordenan candidatos por a_i ascendente (se intenta eliminar primero los “menos usados”). *Nota de lectura de código:* a_i suele aparecer como `RRR_i` y $I_{positivo}$ como `ilocPOS`.

Eta 2.1: NOENF (sin enforcement explícito de positividad)

Para cada candidato propuesto:

1. quitar temporalmente la fila candidata;
2. para cada nodo j , resolver actualización de mínimo cambio:

$$A_{j,keep} \Delta w_j = b_j - A_{j,keep} w_{j,before},$$

$$w_{j,after} = w_{j,before} + \Delta w_j;$$

3. si hay rank deficiency o algún peso negativo ($< -\text{tol_neg}$), se rechaza ese candidato;
4. si todos los nodos son factibles, se acepta la eliminación.

Esto corresponde a una **pasada de eliminación sin enforcement explícito de positividad** (`Loop_MAWecmNOENF_c1-like`).

Eta 2.2: NOENFe local (positividad explícita, sin grafo)

Si NOENF no puede eliminar más, se pasa a active-set local (`Loop_MAWecmNOENFe-like`), todavía **sin** acoplar nodos con grafo:

- se impone no negatividad explícita durante la actualización local;
- se acepta el primer candidato factible según el orden.

Qué sistema lineal se resuelve en NOENF (exactamente)

Para un candidato a eliminar y un nodo j :

$$A_{j,keep} \in \mathbb{R}^{m_j \times r_{keep}}, \quad w_{j,before} \in \mathbb{R}^{r_{keep}}.$$

Se resuelve:

$$A_{j,keep} \Delta w_j = b_j - A_{j,keep} w_{j,before}.$$

En código:

$$\Delta w_j = \text{np.linalg.lstsq}(A_{j,keep}, rhs).$$

Luego:

$$w_{j,after} = w_{j,before} + \Delta w_j.$$

Se rechaza el candidato si:

- `matrix_rank`($A_{j,keep}$) es insuficiente;
- existe entrada negativa por debajo de `-tol_neg`.

Qué sistema se resuelve en NOENFe local (active-set)

En la implementación actual (`mawecm_pruning.py`), por cada nodo j con conjunto libre F_j :

$$A_{j,F_j} \in \mathbb{R}^{m_j \times |F_j|}.$$

Se calcula una solución particular:

$$w_{p,j} = A_{j,F_j}^\top \left(A_{j,F_j} A_{j,F_j}^\top \right)^{-1} b_j.$$

En código no se forma la inversa explícita, se usa:

$$\text{np.linalg.solve}(G_m, b_j), \quad G_m = A_{j,F_j} A_{j,F_j}^\top,$$

y si falla:

$$\text{np.linalg.lstsq}(G_m, b_j).$$

Después se construye base de null-space N_j (vía SVD) y se resuelve el sistema reducido:

$$\left(N^\top H N \right) y = N^\top (g - H z_p), \quad z = z_p + N y.$$

En código:

- intento 1: `scipy.sparse.linalg.spsolve`;
- fallback: `np.linalg.lstsq`.

En la fase 1 sin grafo:

$$\alpha_{smooth} = 0, \quad K_{graph} = 0 \quad \Rightarrow \quad H = I,$$

así que este paso se simplifica bastante.

Condición de parada

Se detiene cuando:

- $|I_{activo}| \leq n_{stop}$ (definido por `-maw-min-support-size` o `auto`), o
- no hay más eliminaciones factibles.

16.6. Paso 3: salida del pruning

El resultado principal es:

$$\mathcal{Z}_{red} \subseteq \mathcal{Z}_{ini}, \quad W_{train} \in \mathbb{R}^{|\mathcal{Z}_{red}| \times N_n}.$$

Además se valida:

$$\max_j \frac{\|A_j w^{(j)} - b_j\|_2}{\|b_j\|_2} \leq \text{strict_constraint_rel_tol}, \quad \min_{i,j} W_{train}(i,j) \geq -\text{strict_negative_tol}.$$

16.7. Paso 4: RBF final de pesos adaptativos

Con pares de entrenamiento:

$$\{q_m^{(j)}, w^{(j)}\}_{j=1}^{N_n},$$

se ajusta RBF multi-output:

$$q_m \mapsto w(q_m) \in \mathbb{R}^{|\mathcal{Z}_{red}|}.$$

En online:

- clipping no negativo (si está activo);
- renormalización opcional a objetivo de suma (por defecto $\sum w_{ini}$).

Cómo se ajusta ese RBF (álgebra + función usada)

En `mawecm_rbf_weights.py`:

- se arma Φ (kernel gaussiano entre muestras y centros);
- si no hay parte polinómica:

$$X = \text{np.linalg.lstsq}(\Phi^\top \Phi + \lambda I, \Phi^\top Y);$$

- si hay parte polinómica (constante o afín), se resuelve el sistema bloque:

$$\begin{bmatrix} \Phi^\top \Phi + \lambda I & \Phi^\top P \\ P^\top \Phi & P^\top P \end{bmatrix} \begin{bmatrix} X_\phi \\ X_{poly} \end{bmatrix} = \begin{bmatrix} \Phi^\top Y \\ P^\top Y \end{bmatrix},$$

también con `np.linalg.lstsq`.

Homogenización por soporte MAW (modo `maw_support_nnls`)

Cuando se ajustan pesos de homogenización sobre soporte MAW reducido, el problema es:

$$\min_{w \geq 0} \|Aw - b\|_2^2 + \lambda \|w\|_2^2.$$

En código **no** se usa `scipy.optimize.nnls` directo; se usa `scipy.optimize.lsqr_linear` con cotas $(0, \infty)$ (NNLS acotado), con secuencia robusta:

1. `method='trf', lsq_solver='lsqr'`;
2. `fallback method='trf', lsq_solver='exact'`;
3. `fallback method='bvls'`.

Si todas fallan, se lanza error explícito.

16.8. Nota importante: “sin grafo” no significa “sin Laplaciano en disco”

El grafo estructurado se puede seguir guardando desde Stage0/Stage8a, pero en esta fase:

`use-global-graph-2ndstage = 0`

y el pruning no acopla nodos por K_{graph} . El acoplamiento por grafo queda para una segunda fase.

17. Modos MAW actuales (implementación 2D)

17.1. `maw-mode=res_only`

- MAW adapta solo residual.
- Homogenización en online puede venir de: `full_mesh`, `fixed_ecm` o `maw_support_nnl`.
- Es el modo más robusto para arrancar y depurar.

17.2. `maw-mode=single_res_hom`

- MAW se entrena con restricciones locales combinadas residual+hom.
- En online, homogenización puede evaluarse con pesos MAW del mismo soporte.
- Si además impones suma de pesos local, el soporte mínimo factible suele subir.

17.3. Aclaración práctica sobre Z_{union}

Cuando residual y homogenización usan soportes distintos:

$$Z_{union} = Z_{res} \cup Z_{\varepsilon} \cup Z_{\sigma}.$$

Por eso puedes pedir, por ejemplo, un tamaño objetivo en residual (p.ej. 100) y terminar con un **union** mayor en la malla HROM final. No es inconsistencia: son objetivos distintos compartiendo una malla reducida final.

18. Stage8 online: qué se compara exactamente

En `stage8_test_hprom_mawecm_rbf_online.py` (o su variante GPR) se ejecuta la misma trayectoria para:

- FOM,
- PROM,
- HPROM-MAWECM.

Se reporta:

- error relativo en strain y stress macroscópicos;
- error en coordenadas de estado:

$$\|q_m^{HPROM} - q_m^{PROM}\|, \quad \|q_s^{HPROM} - q_s^{PROM}\|;$$

- tiempo total y *speedup*.

Si activas malla HROM (`-hprom-use-hrom-mdpa 1`), el costo de ensamblaje cae; si no, aunque haya pesos hiperreducidos, parte del costo de malla completa puede quedarse.

19. Aclaraciones clave (las que más confunden)

19.1. “¿q_pod se usa para resolver online?”

En el flujo actual, la incógnita online es q_m . q_s viene del decoder RBF. q_{pod} puede guardarse como diagnóstico, pero no es la incógnita primaria del Newton.

19.2. “¿Qué es R_r ?”

$$R_r = \left(\frac{\partial u_f}{\partial q_m} \right)^\top R_f.$$

Es el residual en el subespacio reducido. El criterio principal de convergencia en PROM/HPROM está ligado a $\|R_r\|$.

19.3. “¿Por qué aparece K_old ?”

Se reutiliza la rigidez reducida del paso/iteración anterior como aceleración en la primera iteración correctora cuando aplica.

19.4. “¿Qué significa single en ECM/MAW?”

- En ECM clásico Stage6b: `single` = una sola selección/pesos para res+eps+sig.
- En MAW Stage8b: `single_res_hom` = entrenamiento MAW usando restricciones de residual+hom juntas.

20. Ejemplo mini de tamaños (con números tuyos)

En una corrida típica:

$$n_f = 3924, r = 13, d_m = 2, d_s = 11, n_e = 990, N_n = 451.$$

20.1. Decoder

$$\Phi_m \in \mathbb{R}^{3924 \times 2}, A_m \in \mathbb{R}^{2 \times 2}, \Phi_s \in \mathbb{R}^{3924 \times 11}.$$

Para un estado:

$$q_m \in \mathbb{R}^2 \Rightarrow q_s \in \mathbb{R}^{11} \Rightarrow u_f \in \mathbb{R}^{3924}.$$

20.2. MAW data

$$W_{train} \in \mathbb{R}^{|Z_{red}| \times 451}.$$

Si $|Z_{red}| = 172$, entonces:

$$W_{train} \in \mathbb{R}^{172 \times 451}.$$

21. Checklist práctico para depurar

1. Verifica coherencia dimensional de `A_m`, `C_m`, `C_s`, `phi_m`, `phi_s`.
2. Verifica que Stage4 esté en `rbf-input-space=q_m`.
3. Verifica que `q_m_train` usado en Stage3 venga de “consistent” (no LS crudo).

4. En MAW-single, reconstruye Stage8a con `-include-homogenization 1`.
5. En MAW-single online usa `-hprom-homogenization-mode maw_single`.
6. Si usas HROM mdpa, confirma mapeo full $\leftrightarrow$ hrom en el `ecm_weights_all.npz`.

22. Conclusión

Tu implementación actual ya tiene una estructura muy clara:

- una capa de **coordenadas reducidas coherentes** (Stage2a/2b);
- un **decoder** $q_m \mapsto q_s$ por RBF (Stage4);
- un PROM/HPROM con **Newton reducido en q_m** ;
- ECM clásico para baseline;
- MAW-ECM con pesos adaptativos en residual, y ahora también modo single res+hom.

Si quieres, como siguiente documento puedo hacer uno aún más “de pizarra”:

- solo 3 páginas;
- un ejemplo numérico completo con $r = 3$, $d_m = 1$, $d_s = 2$, $n_e = 5$;
- y resolver a mano un paso de Newton reducido + un paso de MAW pruning.